CloudOps 01 Monitoring and Incident Response Detailed Console walkthrough, visual sequence, and completed evidence

Complete this exercise in a dedicated nonproduction AWS account in us-east-1 using synthetic data. No CLI profile is required for the Console-first run.

account	1-day logs	deploy	Errors metric	In alarm	OK

CloudOps-01: Console walkthrough

Region: us-east-1. Estimated study time: 60-90 minutes plus setup. No CLI profile is needed. Labels may vary slightly as the AWS Console changes.

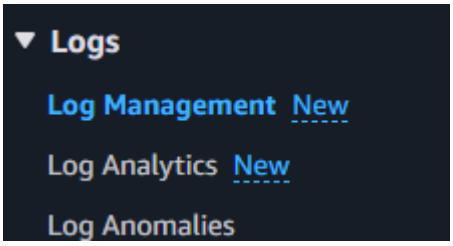
1. Confirm readiness

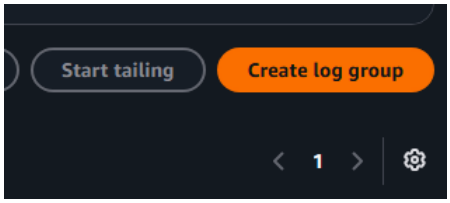
Sign in to a dedicated nonproduction account.

Check the account identity in the Console and select **US East (N. Virginia)** (`us-east-1`).

Record the account and Region in private notes, choose a budget alert and cleanup time, and use synthetic inputs only. No AWS CLI profile is required for this Console-first exercise.

Use cloudops-01-monitoring as the function name, or add a personal suffix consistently. Record resource names and ownership; use Project, Track, Lab, and Owner tags where supported.

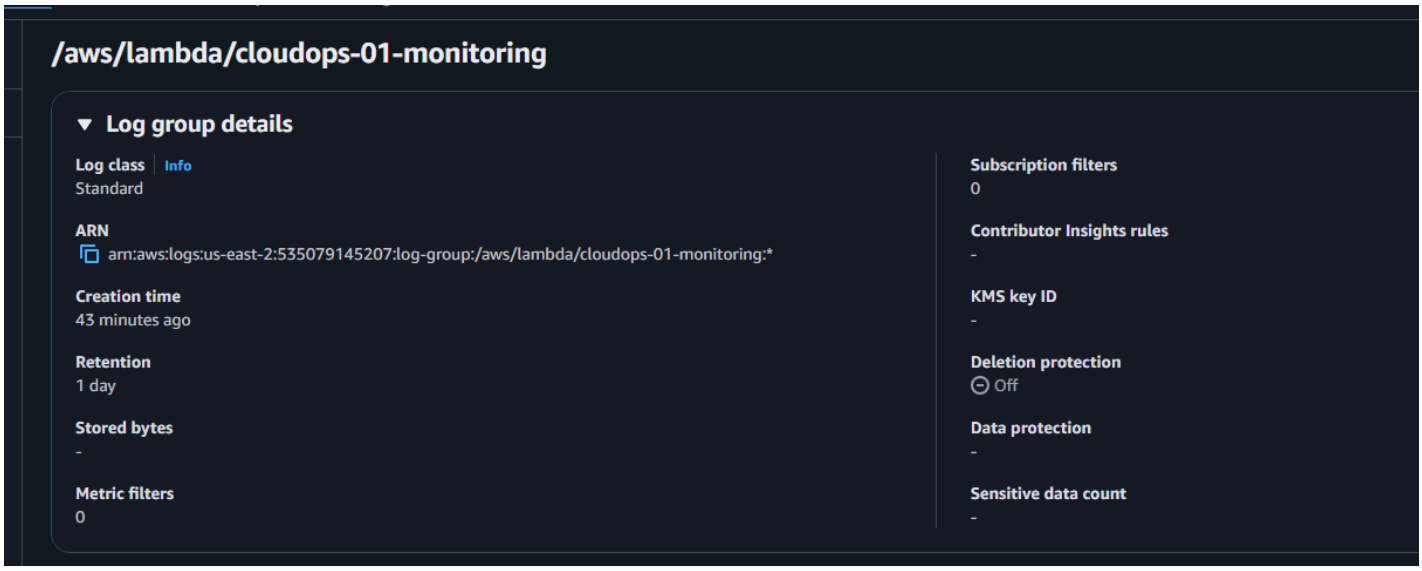




2. Create the log group and execution role

In **CloudWatch → Logs → Log groups**, choose **Create log group**. Enter exactly:

```
/aws/lambda/cloudops-01-monitoring
```



Set **Retention setting** to **1 day**, then create the group. Do not create a group named only `cloudops-01-monitoring`; Lambda uses the `/aws/lambda/` pattern.

In **IAM → Roles**, choose **Create role**, select **AWS service → Lambda**, and create a dedicated role. Attach an inline policy containing only `logs:CreateLogStream` and `logs:PutLogEvents`, scoped to the streams in your lab group:

```
{
 "Version": "2012-10-17",
 "Statement": [{
 "Effect": "Allow",
 "Action": ["logs:CreateLogStream", "logs:PutLogEvents"],
 "Resource": "arn:aws:logs:us-east-1:YOUR_ACCOUNT_ID:log-group:/aws/lambda/cloudops-01-monitoring:*"
}]
}
```



Replace `YOUR_ACCOUNT_ID` and any name suffix.

The group already exists, so the runtime needs no `logs:CreateLogGroup` permission. Lambda trust uses the service principal `lambda.amazonaws.com` and `sts:AssumeRole`.

The learner's own permissions to create and pass the role are separate. If you use a suffix, change the function name, log-group name, policy ARN, and all later metric dimensions consistently.

3. Deploy the tiny workload

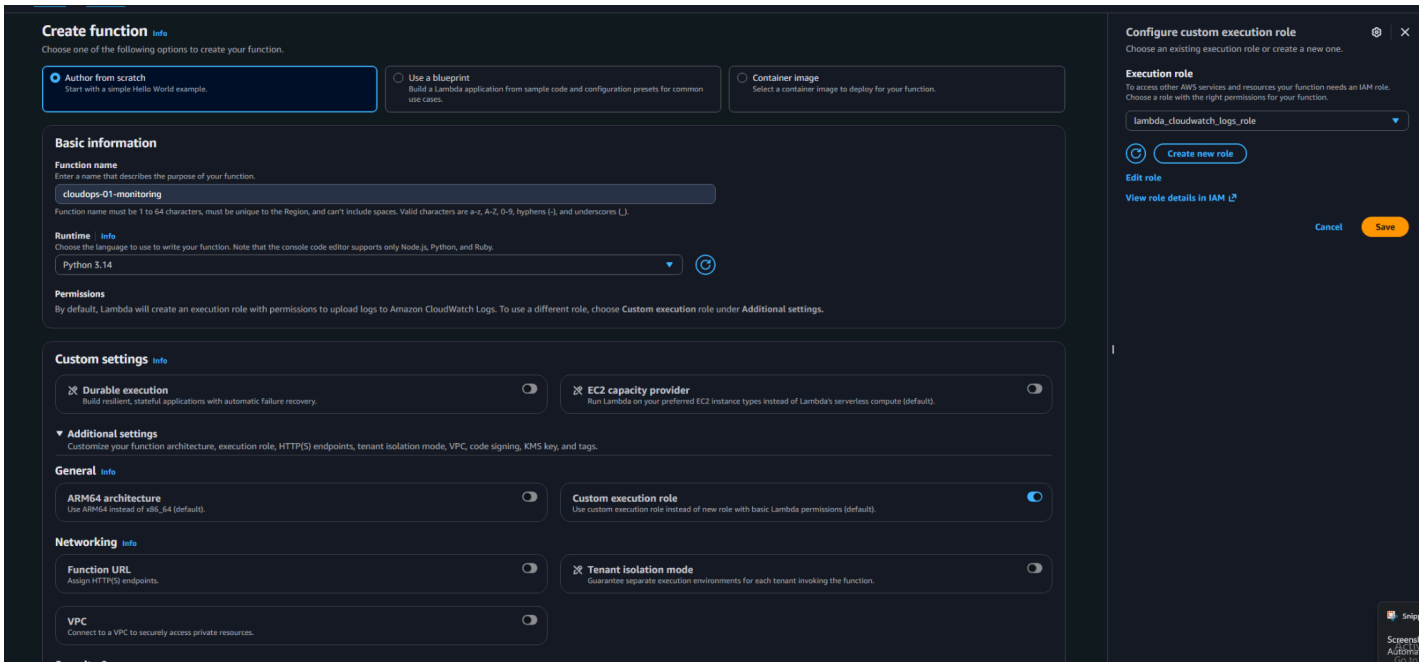
In **Lambda → Functions → Create function**, choose **Author from scratch**:
• Name: cloudops-01-monitoring.

• Runtime: a currently supported Python runtime, such as the current Python 3.x option shown by the

Console. • Execution role: the dedicated role above.

• Memory: 128 MB; timeout: three seconds.

• No function URL, API, VPC attachment, external triggers, or provisioned concurrency.



After the function opens, select the **Code** tab, open `lambda_function.py`, replace its contents with [the handler](src/lambda_function.py), and choose **Deploy**. Keep the handler setting `lambda_function.lambda_handler`. Keep Lambda's default logging configuration; the handler emits structured JSON lines through `print()`.

Handler:

```python
import json


def lambda_handler(event, context):
    fail = event.get("fail", False) is True

    record = {
        "level": "ERROR" if fail else "INFO",
        "service": "cloudops-01",
        "request_id": context.aws_request_id,
        "outcome": "failure" if fail else "success",
        "message": (
            "Deliberate lab failure"
            if fail
            else "Lab invocation succeeded"
        ),
    }

    print(json.dumps(record))

    if fail:
        raise RuntimeError("Deliberate CloudOps lab failure")

    return {
        "ok": True,
        "request_id": context.aws_request_id,
    }
```

Hold Shift to drop

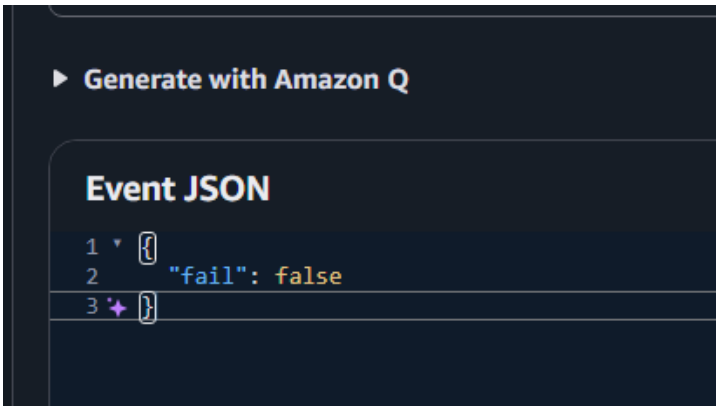
Save two private Console test events:

Success:

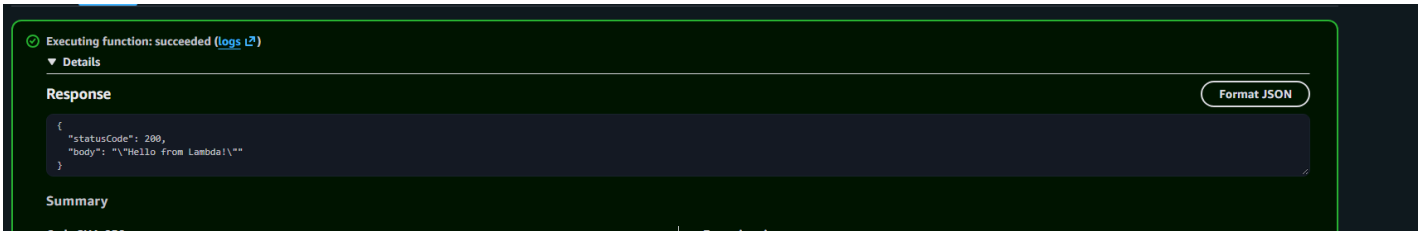
```
{"fail": false}
```

Failure:

```
{"fail": true}
```



Run the **success** event once.



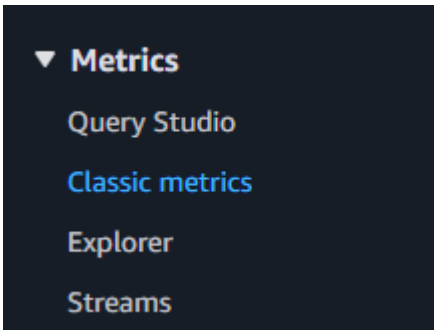
The response should contain `"ok": true` and a `request_id`.

Open **Monitor → View CloudWatch logs**, select the newest stream, and confirm the same request ID appears in a JSON log line with `"outcome": "success"`.

If logs are missing, inspect the group name, Region, execution role, and stream permissions before continuing.

4. Build the dashboard

In **CloudWatch → Metrics → All metrics → Lambda → By Function Name**, select the row whose dimension is exactly `FunctionName = cloudops-01-monitoring`. Do not select an account-wide aggregate or another function/alias. Add these four widgets to a new dashboard named `cloudops-01-monitoring`:



| Metric | Statistic | Period |

| --- | --- | --- |

| Invocations | Sum | 1 minute |

| Errors | Sum | 1 minute |

| Throttles | Sum | 1 minute |

| Duration | Average | 1 minute |

Metrics may take several minutes to appear. Before the first invocation, the graph may show **No data available**; that is expected.

5. Create an Errors alarm

From the selected `Errors` metric, choose **Create alarm**. On **Specify metric and conditions**, use:

• Namespace: `AWS/Lambda`.

• **Metric name: `Errors`**. Do not enter the alarm name here.

• Dimension: `FunctionName = cloudops-01-monitoring`.
• Statistic: `Sum`; period: `1 minute`.

• Threshold type: `Static`; condition: `Greater/Equal`.

• Threshold value: `1`.

• Evaluation: `1 out of 1` datapoints/evaluation periods.

• Additional configuration → missing data: **Treat missing data as not breaching**.

On **Configure actions**, keep the alarm state trigger as **In alarm**. For the optional email extension, choose **Create new topic**, use the topic name `cloudops-01-monitoring-alerts`, and enter an email address you control. Choose **Add notification**. The alarm can work without email, so you may leave this section empty if you do not want the extension.

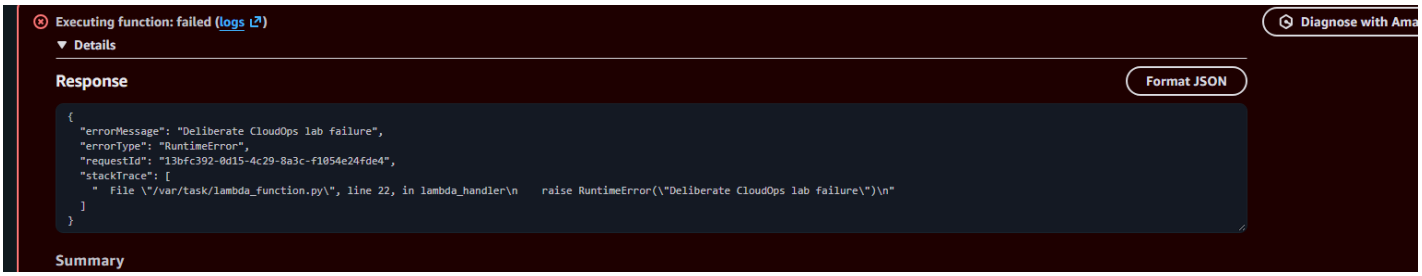
If the Console warns that the SNS endpoint is pending confirmation, open the AWS subscription email and click **Confirm subscription**. Refresh the alarm page and confirm the subscription is no longer pending. A pending subscription does not receive alarm messages.

On **Add alarm details**, set the alarm name to `cloudops-01-monitoring-errors`. The metric remains `Errors`; `cloudops-01-monitoring-errors` is only the alarm name. Optionally add tags such as `Project=content-failures-api`, `Track=CloudOps`, `Lab=cloudops-01-monitoring`, and `Environment=sandbox`.
Create the alarm.

The initial alarm state may be `OK` or `INSUFFICIENT_DATA`. Both can be normal while CloudWatch is waiting for datapoints. Run another success event and allow a few minutes for metric publication and alarm evaluation; do not manually force the alarm state.

6. Cause and investigate the incident

Run the **failure** event three times, synchronously, with no loop or background generator. Stop traffic and wait for metric publication and alarm evaluation.

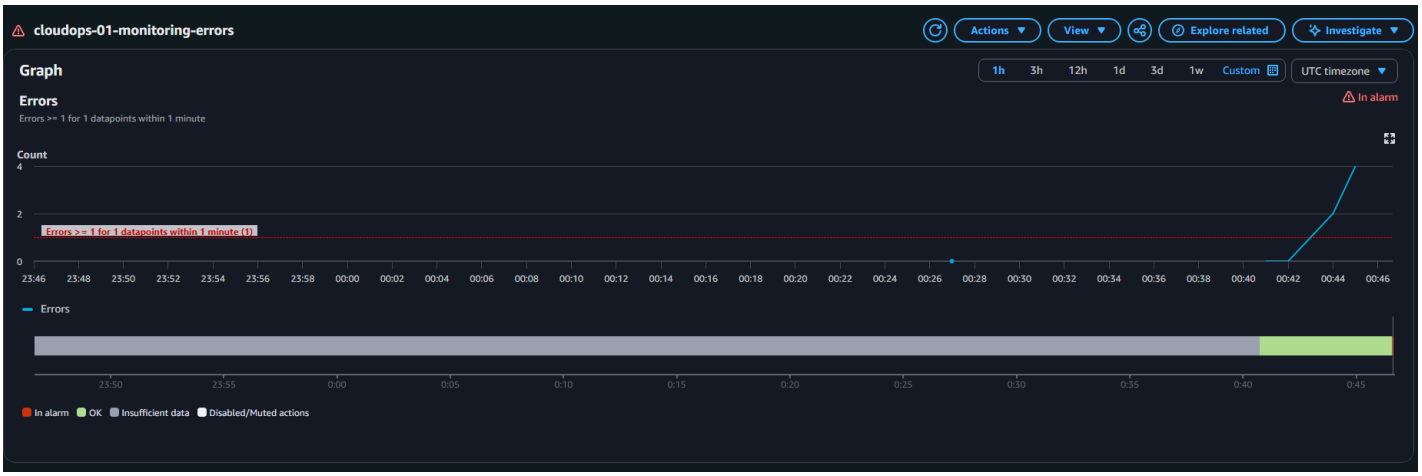


The handler deliberately raises RuntimeError. Returning a JSON object containing statusCode 500 would be a successful Lambda invocation and would not test this native Errors alarm.

Find an ALARM transition in alarm history and examine the triggering datapoint. In Logs Insights, select the lab group and a time window covering your tests:

```
fields @timestamp, @message
| parse @message /"request_id": "(?<lab_request_id>[^"]+)"/
| parse @message /"outcome": "(?<lab_outcome>[^"]+)"/
| filter lab_outcome = "failure"
| sort @timestamp desc
| limit 20
```

Link one failure log to its invocation request ID and explain the source of the error. Inspect Errors, Invocations, and Throttles separately. No throttling is deliberately generated in this first drill.



If `ALARM` does not appear, check the function and Region, metric name (`Errors`), `FunctionName` dimension, `Sum` versus `Average`, time range, period, threshold, and actual raised exceptions. Generate at most one additional failure after correcting the configuration. Do not force the alarm state manually as a substitute.

7. Recover and prove it

Select the success event and invoke three times after the failure period. Verify successful responses and success logs, then observe an eventual OK transition.

Record the ALARM and OK timestamps and the successful recovery requests. Older breaching datapoints can affect evaluation briefly. An alarm configured with notBreaching can also return to OK after traffic stops; idle OK alone does not prove workload recovery.



Optional exercise: edit the alarm and change missing-data treatment to **Missing**, stop traffic, and observe eventual `INSUFFICIENT_DATA` after usable datapoints age out. Restore **Treat missing data as not breaching** and verify the setting.

8. Save evidence and clean up

Copy [the evidence template](../../evidence-template.md) into private notes. Capture the dashboard, alarm

configuration/history, sanitized failure/recovery logs, and optional confirmed notification.

Once those are saved, manually remove only the owned alarm, dashboard, subscription/topic, function, log group, role/policy, and saved test events with the function. Log retention does not remove dashboards or alarms. Recheck resource lists and note any retained resource.

Completion checklist

- [ ] Sandbox account, Region, budget, names, and cleanup time confirmed.
- [ ] Success response and structured log verified.
- [ ] Dashboard shows the correct metrics and dimensions.
- [ ] Actual function errors cause ALARM.
- [ ] Root cause demonstrated through request-correlated logs.
- [ ] Successful recovery invocations verified; subsequent OK recorded.
- [ ] Missing-data behavior explained.
- [ ] Optional SNS notification verified, or marked not selected.
- [ ] Owned resources removed and cleanup verified.

Local fixture verification

From the repository root, with a working Python installation:

```
python -B -m unittest discover -s labs/cloudops/01-monitoring-incidents/tests -v
```

These checks validate local behavior only. AWS permissions, alarm transitions, notifications, and cleanup require the Console exercise.

Completed evidence

CloudOps-01 monitoring and incident response evidence

Date: 2026-10-12 (UTC shown by AWS Console)

Region: us-east-1

Function: `cloudops-01-monitoring`

Alarm: `cloudops-01-monitoring-errors`

Result

- [x] Lambda fixture deployed and executed.
- [x] Deliberate failure raised `RuntimeError: Deliberate CloudOps lab failure`.
- [x] Native `AWS/Lambda` `Errors` metric selected with `Sum`, one-minute period, threshold `>= 1`.
- [x] Alarm entered `In alarm` after recorded errors.
- [x] Successful invocations returned the metric to zero.
- [x] Alarm recovered to `OK`.

• [x] Missing data treatment was `Treat missing data as good (not breaching threshold)`.

• [x] Alarm actions were enabled.

• [ ] SNS email confirmation: record confirmed or not selected.

• [ ] Resource cleanup: record completion after deleting lab resources.

Observed configuration

```
Namespace: AWS/Lambda
Metric name: Errors
FunctionName: cloudops-01-monitoring
Statistic: Sum
Period: 1 minute
Datapoints to alarm: 1 out of 1
Threshold: Errors >= 1
Missing data: Treat missing data as good
```

The alarm initially used the wrong metric name (`cloudops-01-monitoring-errors`) and stayed `OK` because missing data was treated as non-breaching. It was corrected to the native Lambda metric `Errors` before the incident drill.

Failure and recovery evidence

The failing test returned this request ID, which was used to correlate the invocation with its log record:

```
13bfc392-0d15-4c29-8a3c-f1054e24fde4
```

The alarm graph showed the error count above the threshold, then returning to zero. The final Console state was `OK`; the supplied screenshot showed the last state update as `2026-10-12 00:48:36 (UTC)`.

Lessons

• `Errors` is the metric name; `cloudops-01-monitoring-errors` is the alarm name.
• The handler must raise an exception to create a Lambda `Errors` datapoint.

• An SNS subscription must be confirmed before email notifications work.

• Successful recovery invocations prove recovery more strongly than an idle `OK` caused by missing-data treatment.

Cleanup record

• Alarm deleted: [ ]

• Dashboard deleted: [ ]

• SNS topic/subscription deleted: [ ]

• Lambda deleted: [ ]

• Log group deleted: [ ]

• IAM role and inline policy deleted: [ ]

• Retained resources reviewed: [ ]

The chat screenshots showed the Hello World response, the corrected RuntimeError response, the wrong metric-name configuration, the corrected Errors alarm in alarm, and the final recovered OK state. Original chat image attachments were not available as local files for direct embedding; these states are preserved in the visual flow and evidence record.

AWS CLI Equivalent

CloudOps 01 monitoring and incident response

This appendix preserves the original guide and adds the complete PowerShell CLI repeat. Use a dedicated sandbox profile and us-east-1. Do not paste credentials into files.

1. Configure and verify the profile

```
aws configure sso
aws sso login --profile YOUR_PROFILE
aws sts get-caller-identity --profile YOUR_PROFILE
```

2. Set variables

```
$Profile = "YOUR_PROFILE"
$Region = "us-east-1"
$FunctionName = "cloudops-01-monitoring"
$LogGroup = "/aws/lambda/$FunctionName"
$RoleName = "$FunctionName-execution-role"
$AlarmName = "$FunctionName-errors"
$AccountId = aws sts get-caller-identity --profile $Profile --query Account --output text
```

3. Create logs and IAM

```
aws logs create-log-group --log-group-name $LogGroup --region $Region --profile $Profile
aws logs put-retention-policy --log-group-name $LogGroup --retention-in-days 1 --region $Region --profile
$Profile

Create trust-policy.json for lambda.amazonaws.com, then run:
aws iam create-role --role-name $RoleName --assume-role-policy-document file://trust-policy.json --profile
$Profile
aws iam put-role-policy --role-name $RoleName --policy-name "$FunctionName-logging" --policy-document
file://logging-policy.json --profile $Profile
Start-Sleep -Seconds 10
```

4. Package and deploy

```
Compress-Archive -Path labs/cloudops/01-monitoring-incidents/src/lambda_function.py -DestinationPath
cloudops-01-monitoring.zip -Force
$RoleArn = aws iam get-role --role-name $RoleName --query Role.Arn --output text --profile $Profile
aws lambda create-function --function-name $FunctionName --runtime python3.12 --handler
lambda_function.lambda_handler --role $RoleArn --zip-file fileb://cloudops-01-monitoring.zip --memory-size 128
--timeout 3 --region $Region --profile $Profile
```

5. Invoke success and failure

```
"{"fail": false}" | Set-Content success.json
"{"fail": true}" | Set-Content failure.json
aws lambda invoke --function-name $FunctionName --payload fileb://success.json --cli-binary-format raw-in-
base64-out success-response.json --region $Region --profile $Profile
aws lambda invoke --function-name $FunctionName --payload fileb://failure.json --cli-binary-format raw-in-
base64-out failure-response.json --region $Region --profile $Profile
# Run the failure invocation three times.
```

6. Create and inspect the alarm

```
aws cloudwatch put-metric-alarm --alarm-name $AlarmName --namespace AWS/Lambda --metric-name Errors --dimensions
Name=FunctionName,Value=$FunctionName --statistic Sum --period 60 --evaluation-periods 1 --datapoints-to-alarm
1 --threshold 1 --comparison-operator GreaterThanOrEqualToThreshold --treat-missing-data notBreaching --region
$Region --profile $Profile

aws cloudwatch describe-alarms --alarm-names $AlarmName --query
"MetricAlarms[0].[AlarmName,StateValue,MetricName,Namespace,Threshold]" --output table --region $Region
--profile $Profile
```

7. Inspect logs and recover

```
aws logs tail $LogGroup --since 30m --format short --region $Region --profile $Profile
# Invoke success.json three times, then wait for one-minute evaluation.
aws cloudwatch describe-alarms --alarm-names $AlarmName --query
"MetricAlarms[0].[AlarmName,StateValue,StateReason]" --output table --region $Region --profile $Profile
```

8. Clean up

```
aws cloudwatch delete-alarms --alarm-names $AlarmName --region $Region --profile $Profile
aws lambda delete-function --function-name $FunctionName --region $Region --profile $Profile
aws logs delete-log-group --log-group-name $LogGroup --region $Region --profile $Profile
aws iam delete-role-policy --role-name $RoleName --policy-name "$FunctionName-logging" --profile $Profile
aws iam delete-role --role-name $RoleName --profile $Profile
```

Terraform section by file

The following section is included in this PDF in the requested file-by-file order. It is sourced from terraform-guide.md.

CloudOps 01 Terraform implementation

This is a separate sandbox implementation of CloudOps-01. It does not use or modify the enterprise `infra/` root. Use a dedicated directory and state for this lab, a nonproduction account, and `us-east-1`.

The files below are shown in the order to create them. Terraform provisions the resources; the AWS CLI or Console still performs the runtime failure and recovery drill.

1. Directory layout

Create this layout from the repository root:

```
labs/cloudops/01-monitoring-incidents/terraform/
  versions.tf
  providers.tf
  variables.tf
  locals.tf
  iam.tf
  logs.tf
  lambda.tf
  cloudwatch.tf
  outputs.tf
  terraform.tfvars.example
```

The Lambda source remains here:

```
labs/cloudops/01-monitoring-incidents/src/lambda_function.py
```

2. `versions.tf`

Create `terraform/versions.tf`:

```
terraform {
  required_version = ">= 1.9.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
    archive = {
      source  = "hashicorp/archive"
      version = "~> 2.4"
    }
  }
}
```

3. `providers.tf`

Create `terraform/providers.tf`:

```
provider "aws" {
  region = var.aws_region

  default_tags {
    tags = {
      Project     = var.project_name
      Track       = "CloudOps"
      Lab         = "cloudops-01-monitoring"
      Environment = var.environment
      ManagedBy   = "Terraform"
      Owner       = var.owner
    }
  }
}
```

The AWS provider uses the credentials selected by your AWS CLI profile or IAM Identity Center session. Do not add access keys to Terraform files.

4. `variables.tf`

Create `terraform/variables.tf`:

```
variable "aws_region" {
  type        = string
  description = "AWS Region for the lab."
  default     = "us-east-1"
}

variable "project_name" {
  type        = string
  description = "Stable project name."
  default     = "content-failures-api"
}

variable "environment" {
  type        = string
  description = "Sandbox environment label."
  default     = "sandbox"

  validation {
    condition     = contains(["sandbox", "test"], var.environment)
    error_message = "Use sandbox or test for this exercise."
  }
}

variable "owner" {
  type        = string
  description = "Person responsible for cleanup."
}

variable "function_name" {
  type        = string
  description = "Lambda function name."
  default     = "cloudops-01-monitoring"
}
```

5. `locals.tf`

Create `terraform/locals.tf`:

```
locals {
  log_group_name = "/aws/lambda/${var.function_name}"
  role_name      = "${var.function_name}-execution-role"
  alarm_name     = "${var.function_name}-errors"
  source_file    = "${path.module}/../src/lambda_function.py"
}
```

6. `iam.tf`

Create `terraform/iam.tf`:

```
resource "aws_iam_role" "lambda" {
  name = local.role_name

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Principal = {
        Service = "lambda.amazonaws.com"
      }
      Action = "sts:AssumeRole"
    }]
  })
}

resource "aws_iam_role_policy" "lambda_logs" {
  name = "${var.function_name}-logging"
  role = aws_iam_role.lambda.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Effect = "Allow"
      Action = [
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ]
      Resource = "${aws_cloudwatch_log_group.lambda.arn}:*"
    }]
  })
}
```

The policy intentionally omits `logs:CreateLogGroup` because Terraform creates the group first.

7. `logs.tf`

Create `terraform/logs.tf`:

```
resource "aws_cloudwatch_log_group" "lambda" {
  name              = local.log_group_name
  retention_in_days = 1
}
```

8. `lambda.tf`

Create `terraform/lambda.tf`:

```
data "archive_file" "lambda" {
  type        = "zip"
  source_file = local.source_file
  output_path = "${path.module}/lambda.zip"
}

resource "aws_lambda_function" "monitoring" {
  function_name    = var.function_name
  role             = aws_iam_role.lambda.arn
  runtime          = "python3.12"
  handler          = "lambda_function.lambda_handler"
  filename         = data.archive_file.lambda.output_path
  source_code_hash = data.archive_file.lambda.output_base64sha256
  memory_size     = 128
  timeout         = 3

  depends_on = [aws_cloudwatch_log_group.lambda]
}
```

If the AWS Console shows a different currently supported Python runtime, use that runtime consistently in the plan and record the choice.

9. `cloudwatch.tf`

Create `terraform/cloudwatch.tf`:

```
resource "aws_cloudwatch_metric_alarm" "errors" {
  alarm_name          = local.alarm_name
  alarm_description   = "Lambda errors for the CloudOps-01 monitoring lab."
  namespace           = "AWS/Lambda"
  metric_name         = "Errors"
  dimensions          = { FunctionName = aws_lambda_function.monitoring.function_name }
  statistic           = "Sum"
  period              = 60
  evaluation_periods  = 1
  datapoints_to_alarm = 1
  threshold           = 1
  comparison_operator = "GreaterThanOrEqualToThreshold"
  treat_missing_data  = "notBreaching"
}
```

The metric name is `Errors`. The alarm name is `cloudops-01-monitoring-errors`; they are different values.

10. `outputs.tf`

Create `terraform/outputs.tf`:

```
output "function_name" {
  value = aws_lambda_function.monitoring.function_name
}

output "function_arn" {
  value = aws_lambda_function.monitoring.arn
}

output "log_group_name" {
  value = aws_cloudwatch_log_group.lambda.name
}

output "alarm_name" {
  value = aws_cloudwatch_metric_alarm.errors.alarm_name
}

output "execution_role_name" {
  value = aws_iam_role.lambda.name
}
```

11. `terraform.tfvars.example`

Create `terraform/terraform.tfvars.example`:

```
aws_region   = "us-east-1"
project_name = "content-failures-api"
environment  = "sandbox"
owner        = "YOUR_NAME"
```

Copy it to a local `terraform.tfvars`, replace `YOUR_NAME`, and do not commit the local file if it contains personal or account-specific values.

12. Initialize and validate

From `labs/cloudops/01-monitoring-incidents/terraform`:

```
Copy-Item terraform.tfvars.example terraform.tfvars
# Edit terraform.tfvars and replace YOUR_NAME.

terraform init
terraform fmt -check
terraform validate
terraform plan -out cloudops-01.tfplan
```

Review the plan. Verify the account, Region, names, IAM policy, log retention, alarm metric, and resource count before applying.

13. Apply and perform the runtime drill

Apply only after reviewing the sandbox plan:

```
terraform apply cloudops-01.tfplan
terraform output
```

Use the [AWS CLI equivalent](cli-equivalent.md) to invoke `fail=true`, confirm `ALARM`, invoke `fail=false`, and confirm `OK`. Terraform does not replace runtime failure testing.

14. Destroy and verify cleanup

```
terraform plan -destroy -out cloudops-01-destroy.tfplan
terraform apply cloudops-01-destroy.tfplan
```

Confirm the plan contains only this lab's resources. Verify that the Lambda, log group, alarm, role, policy, and generated `lambda.zip` are removed as intended. Never commit `.terraform`, `terraform.tfstate`, `terraform.tfstate.backup`, `terraform.tfvars`, credentials, or secrets.

Important: the CloudWatch metric name is Errors. The alarm name is cloudops-01-monitoring-errors. The original guide pages, including their images, are preserved unchanged before this appendix.